

Public Key Encryption

Recall encryption security game: Attacker can query the oracle as much as they like, then they are tested with messages m_0, m_1 . The Attacker must try to determine which message was encrypted by the scheme (the other message being random). In this case, deterministic functions are insecure since the attacker can know when a message is encrypted if it is the same as a previously queried one.

Public Key IND-CPA Security

Same sec game as before. The attacker chooses two messages of the same length, and is challenged to correctly guess which of these messages was encrypted by the challenger.

The difference is that the attacker does not need an oracle access to the encryption oracle!

Since the public encryption key is known to everyone (including the adversary), and can use it to encrypt any message they want.

The DH Encryption Scheme

Alice knows d_A (private key), g , and p (generator and safe prime)

Alice sends e_A (Alice's public key), Bob returns:

$$g^b \mod p, m \oplus [(e_A)^b \mod p]$$

Where b is some secret value Bob chooses, which is fresh (therefore a randomized encryption scheme: not deterministic!).

Note $e_A = g^{d_A} \mod p$

$$\begin{aligned} m &= c_1 \oplus (g^b)^{d_A} \\ &= (m \oplus (e_A)^b) \oplus (g^{d_A})^b \\ &= m \oplus (e_A)^b \oplus (e_A)^b \end{aligned}$$

May not be secure! Believed to be secure under CDH (Computational Diffie Hellman, g^a, g^b, g^{ab} and g^a, g^b, g^c are indistinguishable) assumption, however this is not always true! g^{ab} may reveal some bits.

Solution: Hash

Hashed DH Encryption Scheme Secure if $h(g^{b*d_A} \mod p)$ is pseudorandom (so the hash function must be a randomness-extractor hash function)

$c_0, c_1: (g^b \mod p, m \oplus h((e_A)^b \mod p))$

ElGamal Public Key Encryption

To encrypt message m to Alice, whose public key is $e_A = g^{d_A} \mod p$:

Bob selects random b , and sends $g^b \mod p, m * (e_A)^b = m * g^{b*d_A} \mod p$

Like DH, but multiply instead of XOR.

Correctness:

$$\begin{aligned} D_{d_A}(g^b \mod p, m * e_A^b \mod p) &= \\ &= [(g^b \mod p)^{-d_A} * (m * (g^{d_A})^b \mod p)] \mod p \\ &= [g^{-b*d_A} * m * g^{b*d_A}] \mod p \\ &= m \end{aligned}$$

ElGamal Public Key Cryptosystem

Problem: $g^{b*d_A} \mod p$ may leak bit(s)

Classical DH solution: use a randomness extractor function as $h(g^{b*d_A} \mod p)$

El-Gamal's solution: use a group where DDH believed to hold (note: message must be encoded as member of the group)

ElGamal is homomorphic

ElGamal PKC: homomorphism

Multiplying two ciphertexts produces a ciphertext of the multiplication of the two plaintexts

$$Mult((x_1, y_1), (x_2, y_2)) = (x_1, x_2, y_1, y_2)$$

RSA Public Key Encryption

Select two large primes p, q ; let $n = pq$ (guaranteed factors of just $1, n, p, q$, hard due to the factoring problem)

Select prime e (public key: (n, e)), or co-prime with $\Phi(n) = (p-1)(q-1)$

Let private key be $d = e^{-1} \bmod \Phi(n)$ (i.e. $ed = 1 \bmod \Phi(n)$)

Encryption: $RSA.E_{e,n}(m) = m^e \bmod n$

Decryption: $RSA.D_{d,n}(c) = c^d \bmod n$ (raising ciphertext to private key)

Correctness: $D_{d,n}(E_{e,n}(m)) = (m^e)^d = m^{ed} = m \bmod n$

Intuitively, $ed = 1 \bmod \Phi(n) \rightarrow m^{ed} = m \bmod n$

Textbook RSA is deterministic and therefore not CPA secure

RSA problem: Find m , given (n, e) and ciphertext value $c = m^e \bmod n$

Padded RSA

Pad and Unpad functions: $m = Unpad(Pad(m; r))$

$$c = [Pad(m, r)]^e \bmod n$$

$$m = Unpad(c^d \bmod n)$$

Adds randomization to prevent detection of repeating plaintext

Padding must be done carefully, certain padding algorithms still do not guarantee CPA security